

Project: ES5 → ES6+ Student Management System Refactor

Project Scenario

Tumhare paas ek purana company ka Student Management System hai jo ES5 mein likha gaya hai.

Tum naye developer ho jisko is codebase ko modern ES6+ mein migrate karna hai.

Ye exactly wahi kaam hai jo real companies mein hota hai — legacy code modernization.

Har step mein ek specific ES6 feature apply karni hai — step by step refactor karo.

Step 0 — Starting Point: Purana ES5 Code

Ye original codebase hai. Copy karo aur students.js file mein save karo:

```
// students.js – ORIGINAL ES5 code (before refactoring)

var students = [
  { name: 'Rahul', marks: [85, 90, 78], city: 'Delhi' },
  { name: 'Priya', marks: [92, 88, 95], city: 'Mumbai' },
  { name: 'Arjun', marks: [70, 75, 68], city: 'Pune' }
];

function getAverage(marks) {
  var total = 0;
  for (var i = 0; i < marks.length; i++) {
    total = total + marks[i];
  }
  return total / marks.length;
}

function generateCard(student) {
  var avg = getAverage(student.marks);
  return 'Name: ' + student.name + ', City: ' + student.city +
    ', Avg: ' + avg;
}

function getToppers(students, minAvg) {
  var result = [];
  for (var i = 0; i < students.length; i++) {
    if (getAverage(students[i].marks) >= minAvg) {
      result.push(students[i]);
    }
  }
  return result;
}
```

Step 1 — var → const / let (Scope Fix)

Sabse pehle sabhi var ko replace karo. Rule: reassign na ho to const, loop variables mein let.

```
const students = [ // array reassign nahi hoga – const
  { name: 'Rahul', marks: [85, 90, 78], city: 'Delhi' },
  { name: 'Priya', marks: [92, 88, 95], city: 'Mumbai' },
  { name: 'Arjun', marks: [70, 75, 68], city: 'Pune' }
];

function getAverage(marks) {
  let total = 0; // loop variable – let
  for (let i = 0; i < marks.length; i++) {
    total += marks[i];
  }
  return total / marks.length;
}
```

Step 2 — function → Arrow Functions + Array Methods

Sab functions arrow functions mein convert karo. for loops ko .reduce() aur .filter() se replace karo.

```
// getAverage – loop to reduce
const getAverage = (marks) =>
  marks.reduce((total, m) => total + m, 0) / marks.length;

// getToppers – loop to filter + default param!
const getToppers = (students, minAvg = 80) =>
  students.filter(s => getAverage(s.marks) >= minAvg);
```

Step 3 — Template Literals + Destructuring

String concatenation hatao aur destructuring add karo function parameters mein.

```
const generateCard = ({ name, city, marks }) => { // destructuring!
  const avg = getAverage(marks).toFixed(1);
  const grade = avg >= 90 ? 'A+' : avg >= 75 ? 'A' : 'B';
  return `${name} (${city}) -> Avg: ${avg}% | Grade: ${grade}`;
};
```

Step 4 — Rest/Spread (Final Touches)

Spread se immutable array operations add karo.

```
// Spread se naya student add karo – original unchanged
const addStudent = (list, newStudent) => [...list, newStudent];

// Update student immutably
const updateMarks = (list, name, newMarks) =>
  list.map(s => s.name === name ? { ...s, marks: newMarks } : s);
```

FINAL — Complete Refactored Code

```
// students.js — FINAL ES6+ Refactored Version

const students = [
  { name: 'Rahul', marks: [85, 90, 78], city: 'Delhi' },
  { name: 'Priya', marks: [92, 88, 95], city: 'Mumbai' },
  { name: 'Arjun', marks: [70, 75, 68], city: 'Pune' }
];

const getAverage = (marks) =>
marks.reduce((total, m) => total + m, 0) / marks.length;

const generateCard = ({ name, city, marks }) => {
  const avg = getAverage(marks).toFixed(1);
  const grade = avg >= 90 ? 'A+' : avg >= 75 ? 'A' : 'B';
  return `${name} (${city}) -> Avg: ${avg}% | Grade: ${grade}`;
};

const getToppers = (students, minAvg = 80) =>
students.filter(s => getAverage(s.marks) >= minAvg);

const addStudent = (list, newStudent) => [...list, newStudent];

const updateMarks = (list, name, newMarks) =>
list.map(s => s.name === name ? { ...s, marks: newMarks } : s);

// Run it!

students.forEach(s => console.log(generateCard(s)));
console.log('Toppers:', getToppers(students).map(s => s.name));

// Output:
// Rahul (Delhi) -> Avg: 84.3% | Grade: A
// Priya (Mumbai) -> Avg: 91.7% | Grade: A+
// Arjun (Pune) -> Avg: 71.0% | Grade: B
// Toppers: ['Rahul', 'Priya']
```

Bonus Challenges (Try karo!)

1. addStudents(...newStudents) — rest operator se multiple students ek saath add karo.
2. Template literal se HTML table generate karo jisme sabke cards rows hoon.
3. getStats() function banao jo { highest, lowest, classAvg } destructure karke return kare.
4. Ek updateCity(list, name, newCity) function banao using spread (immutable).